

CSE 2050: Lab 1: Getting Started

Welcome to Lab 1! Quick notes:

- You'll do labs with a partner in your lab section.
- 4 Participation credits each lab =
 - show up on time,
 - work with your partner,
 - stay engaged until you finish or lab ends,
 - and don't start early (so you and your partner can work together with TA help).
- Coding labs practice: file/folder organization, writing & running Python, and submitting to Gradescope.

Goal: confirm your Python + IDE setup, learn the course folder structure, and make your first Gradescope submission.

What you'll do today

- Install or confirm **Python 3.x** is installed and working.
- Install or confirm an IDE (**VS Code** recommended).
- Create the course folder structure you'll use all semester.
- Create (or join) a **Gradescope** account for this course.
- **Create two small Python files from scratch** and submit to Gradescope.

Required software + links

- [Python 3.x](#)
- [VS Code](#)
- Gradescope: Access Gradescope from HuskyCY.

Part A: Verify Python works

1. Open a terminal
2. Check your Python version:
 - `python --version` or `python3 --version` (common on macOS/Linux). You should see **Python 3.x**.

Part B: Folder structure

All modern operating systems use a hierarchal file structure. We can think of this as a tree - the top level directory, often referred to as **root**, contains all the files and folders on your computer. The exact names of various folders changes based on your operating system, but the general structure is a root directory which contains both the files necessary for various programs to run and the files created and used by individual users. Start by create this structure (best practice for organization all semester):

```
CSE2050/  
  labs/  
    lab01/
```

```
lab02/  
lab03/  
...
```

Rules of thumb

- Each lab goes in its own folder: `lab01`, `lab02`, etc.
- Put your files for that lab inside that lab folder.

Part C: IDE check

- Open your IDE and **open the folder** `CSE2050/labs/lab01` (open the folder, not just a single file).
- Create **two files** in `lab01`:
 - `hello.py`
 - `lab1.py`

STOP - ☒ at this point, you and your partner should be able to navigate the file structure, including creating files.

Part D: Gradescope setup + submission check

You'll use Gradescope all semester. Today's goal is to confirm your setup works end-to-end.

1. Access Gradescope from HuskyCT and Create an account.
2. Join the course.
3. Open the Lab 1 assignment.

Part E: Coding tasks

Task 1: Create `hello.py`

In `labs/CSE2050/lab01`, create a new file named `hello.py`.

Requirements:

- Define a function named `say_hi`
- It takes **no parameters**
- It **returns** the exact string:
 - `Hello, world`. Check [Mini Python Reference](#) for more information about `return` and `print`

Grading behavior

- Submit `hello.py` to Gradescope → you should see **about 10/100**.
(This is a "pipeline check" to confirm your submission works.)

Task 2: Create `lab1.py`

In the same folder, create a new file named `lab1.py`.

Requirements:

- Define a function named `generic_hi`
- It has **one parameter** named `name` with default value `"world"`. Make sure your function has a default value ("world") so it still works even if no name is passed in. If you're not sure how, google "Python default function parameter" and follow an example, being able to quickly find and use the right info is a big part of programming and developing critical skill for computer scientists. If you get stuck, ask your lab partner, a nearby group, your TA, or post in the course Discord.
- It **returns** a greeting string that includes the name and ends with `!`. Check [Mini Python Reference](#) for more information about formatted strings.
- Expected behavior examples:
 - Called with no argument → `Hello, world!`
 - Called with `name="Ada"` → `Hello, Ada!`
 - Called with `name="CSE2050"` → `Hello, CSE2050!`
- Add a few **assert statements** at the bottom of `lab1.py` to test the three cases above.

Now submit **both** `hello.py` and `lab1.py` → you should see **100/100**, if not, check your code and make sure you have no errors. **You can submit as many times as you want before the due date.**

STOP - ☒ at this point, you and your partner should be able to write a basic python script, run it, and see its output. Make sure you are comfortable with the above (ask classmates or the TA if you need assistance).

Mini Python Reference

1) `print` vs `return`

`print(...)`

- Shows text on the screen (terminal).
- Useful for debugging and seeing intermediate values.

```
def demo_print():
    print("This shows up in the terminal")

demo_print() #Expected output: This shows up in the terminal
```

`return ...`

- Sends a value back to whoever called the function.
- Autograders usually check **return values**, not printed text.

```
def demo_return():
    return "This value goes back to the caller"

x = demo_return()
print(x) # Expected output: This value goes back to the caller
```

2) f-strings (formatted strings) An **f-string** is a convenient way to build strings that include variable values.

```
name = "Ada"
message = f"Hello, {name}!"
print(message) #Expected output:Hello, Ada!

items = 3
print(f"You have {items} new messages.") #Expected output:You have 3 new messages.
```

3) assert statements (quick self-tests) An **assert** checks that something is true.

```
x = "hello"
#if condition returns True, then nothing happens:
assert x == "hello"

#if condition returns False, AssertionError is raised:
assert x == "goodbye"
```

How to use asserts in labs: write a few asserts at the bottom of your file to test your function locally before submitting. For example:

```
assert generic_hi('John') == 'Hello, John!'
```

Part F: Conceptual Quiz

Each module has a conceptual quiz. These quizzes are worth 5% of your total grade. You can find these quizzes in HuskyCT.

- Each quiz allows 4 attempts
- After each attempt, you can see which questions you missed
- Quizzes will typically be available by the day before labs. It's a good idea to try them at least once before lab, to ensure you have the appropriate conceptual background.

External Modules

Don't use any library imports (like math, collections, etc.) to implement the required functionality. It's fine to use imports in your testing code, and it's also fine to import your own files/modules (for example, a data structure you implemented yourself).

Submission rules

STOP - ☒ Make sure both you and your partner submit the files so you both get credit for the lab. Before you go, make sure to backup your files using a cloud service like Onedrive.

At a minimum, submit the following files for this lab:

- `hello.py`
- `lab1.py` File names must be exactly: `hello.py` and `lab1.py`
- Resubmit as many times as needed before the due date Students must submit by the due date (typically, Friday at 11:59 pm EST).

☒ **Nice work finishing Lab 1!**

You've now got Python and your IDE set up, your folder structure organized, and you've successfully submitted to Gradescope. These are the tools you'll use for every lab this semester.

And just as important, you practiced working with a partner by talking through ideas, debugging together, and coordinating submissions. Clear communication is a real CS skill, so keep explaining your thinking, asking questions, and checking in with each other. See you in Lab 2!